Exp 12a: Implementation of Echo Client Server Using TCP/UDP Sockets

AIM :

To implement an echo client–server application using TCP and UDP sockets in Java, in which the server sends back to the client every message it receives.

ALGORITHM :

TCP Echo Server

1. Start the experiment.
2. Create a `ServerSocket` on the given port (default 5000) and wait for clients.
3. Accept a client connection using `accept()` and create the input and output streams of the socket.
4. Read a line sent by the client using `readLine()`, display it on the server console and send it back prefixed with “Echo: ”.
5. If the received line is “bye”, close the client connection and wait for the next client.
6. Stop the experiment.

TCP Echo Client

1. Create a `Socket` connected to the server host (default localhost) and port (default 5000).
2. Read a message from the keyboard and send it to the server.
3. Receive the echoed message from the server and display it.
4. Repeat steps 2–3 until the user types “bye”, then close the connection.

UDP Echo Server

1. Create a `DatagramSocket` on port 6000 and a 1024-byte receive buffer.
2. Receive a `DatagramPacket` from a client using `receive()` and display the message.
3. Build a reply packet containing “Echo : ” + message and send it to the address and port of the sender using `send()`.
4. Repeat steps 2–3 indefinitely.

UDP Echo Client

1. Create a `DatagramSocket` and get the server address (localhost).
2. Read a message from the keyboard, put it in a datagram packet and send it to the server at port 6000.
3. Receive the reply packet from the server and display the echoed message.
4. Repeat steps 2–3 for every message entered.

CODE :

TCPEchoServer.java

```
import java.io.*;
import java.net.*;

/**
 * Simple TCP Echo Server.
 * Accepts a client connection, reads lines of text, and echoes them back.
 * Type "bye" from the client to close that connection.
 */
public class TCPEchoServer {
    public static void main(String[] args) throws IOException {
        int port = args.length > 0 ? Integer.parseInt(args[0]) : 5000;

        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("TCP Echo Server listening on port " + port);

            while (true) {
                try (Socket clientSocket = serverSocket.accept()) {
                    System.out.println("Client connected: " + clientSocket.getRemoteSocketAddress());

                    BufferedReader in = new BufferedReader(
                            new InputStreamReader(clientSocket.getInputStream()));
                    PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);

                    String line;
                    while ((line = in.readLine()) != null) {
                        System.out.println("Received: " + line);
                        out.println("Echo: " + line);
                        if (line.equalsIgnoreCase("bye")) {
                            break;
                        }
                    }
                    System.out.println("Client disconnected: " + clientSocket.getRemoteSocketAddress());
                } catch (IOException e) {
                    System.out.println("Connection error: " + e.getMessage());
                }
            }
        }
    }
}
```

TCPEchoClient.java

```
import java.io.*;
import java.net.*;

/**
 * Simple TCP Echo Client.
 * Connects to the TCP echo server, sends lines typed on the console,
 * and prints back what the server echoes. Type "bye" to quit.
 */
public class TCPEchoClient {
    public static void main(String[] args) throws IOException {
        String host = args.length > 0 ? args[0] : "localhost";
        int port = args.length > 1 ? Integer.parseInt(args[1]) : 5000;

        try (Socket socket = new Socket(host, port);
             PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
             BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
             BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in))) {

            System.out.println("Connected to TCP Echo Server at " + host + ":" + port);
            System.out.println("Type messages to send (type 'bye' to quit):");

            String userInput;
            while ((userInput = stdIn.readLine()) != null) {
                out.println(userInput);
                String response = in.readLine();
                System.out.println("Server: " + response);
```

```
                if (userInput.equalsIgnoreCase("bye")) {
                    break;
                }
            }
        }
        System.out.println("Connection closed.");
    }
}
```

UDPEchoServer.java

```
import java.net.*;

class UDPEchoServer {
    public static void main(String[] args) throws Exception{
        DatagramSocket s = new DatagramSocket(6000);
        byte[] b = new byte[1024];
        System.out.println("Server started ...");

        while(true){
            DatagramPacket p = new DatagramPacket(b, b.length);
            s.receive(p);
            String m = new String(p.getData(),0,p.getLength());
            System.out.println("Received: " + m);
            byte[] response = ("Echo : " + m).getBytes();
            s.send(new DatagramPacket(response,response.length,p.getAddress(),p.getPort()));

        }

    }
}
```

UDPEchoClient.java

```
import java.io.*;
import java.net.*;

class UDPEchoClient {
    public static void main(String[] args) throws Exception{
        DatagramSocket s = new DatagramSocket();
        InetAddress a = InetAddress.getByName("localhost");
        BufferedReader in = new BufferedReader(new InputStreamReader(System.in));

        while(true){
            System.out.print("Enter message: ");
            String m = in.readLine();
            byte[] b = m.getBytes();
            DatagramPacket p = new DatagramPacket(b,b.length,a,6000);
            s.send(p);
            byte[] response = new byte[1024];
            p = new DatagramPacket(response,response.length);
            s.receive(p);
            m = new String(p.getData(),0,p.getLength());
            System.out.println(m);
        }
    }
}
```

OUTPUT :

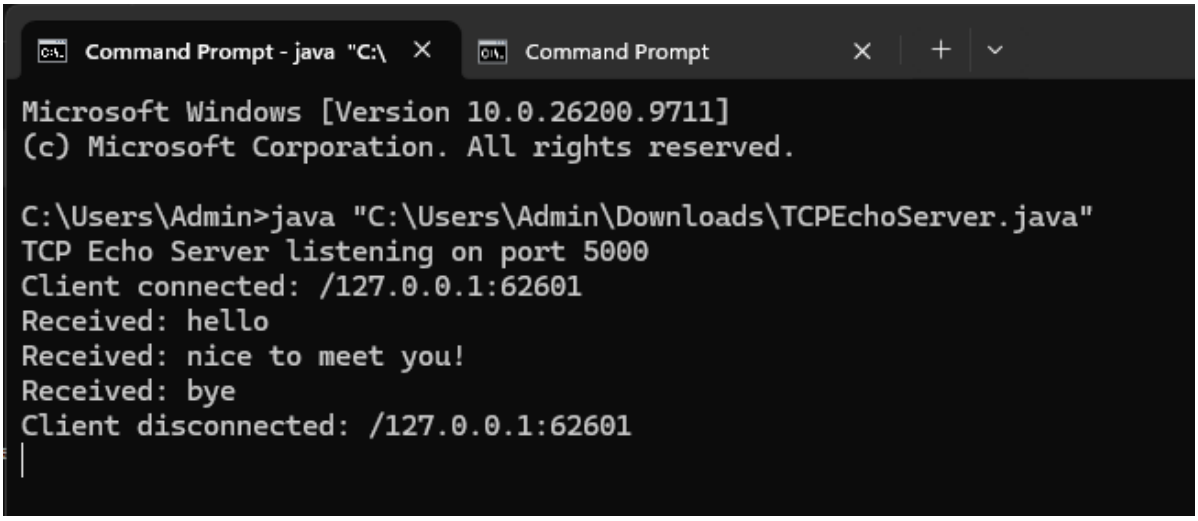


Fig 1. TCP Echo Server – client connected, messages received and client disconnected

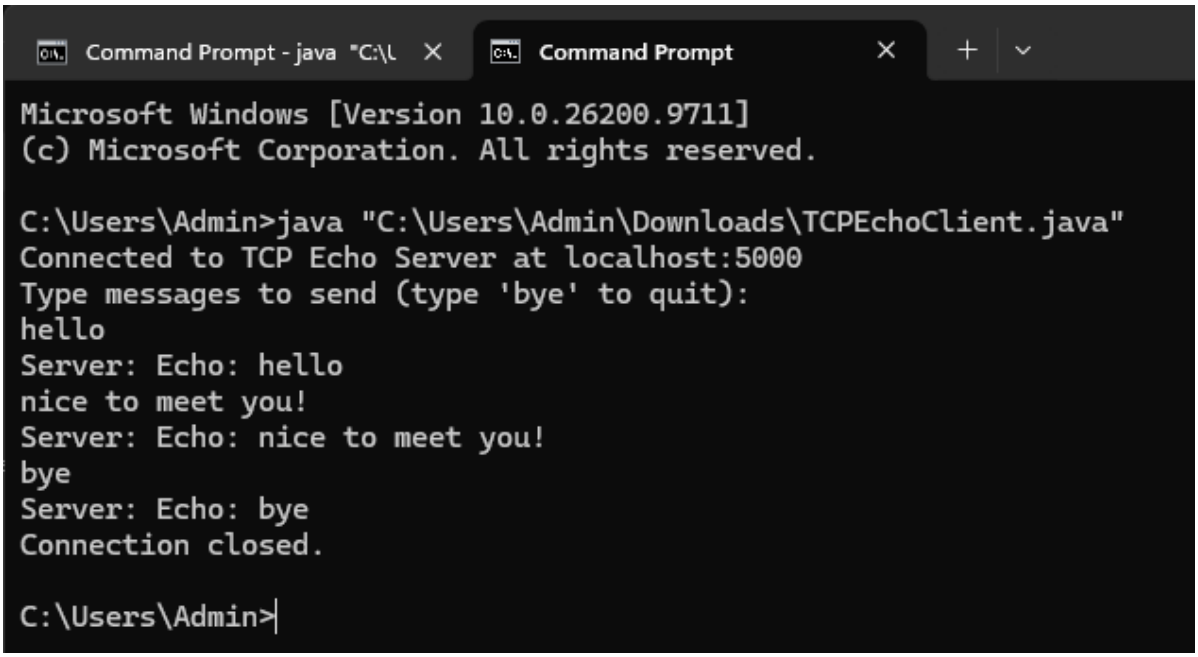


Fig 2. TCP Echo Client – messages sent and echoed back by the server

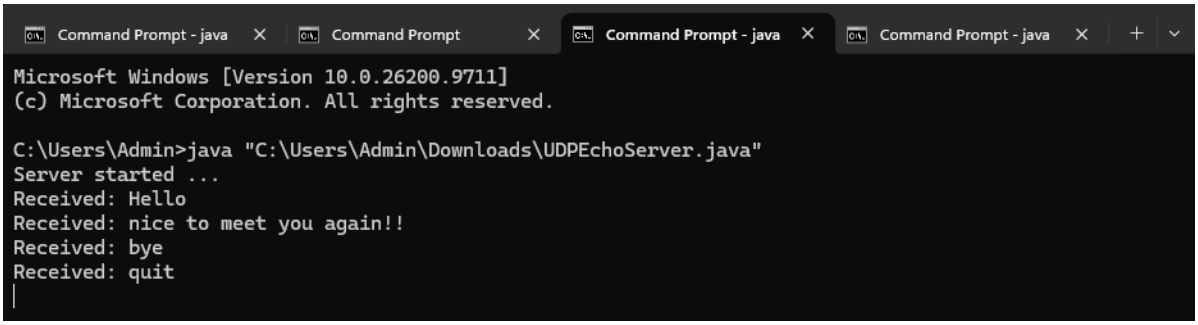


Fig 3. UDP Echo Server – datagrams received from the client

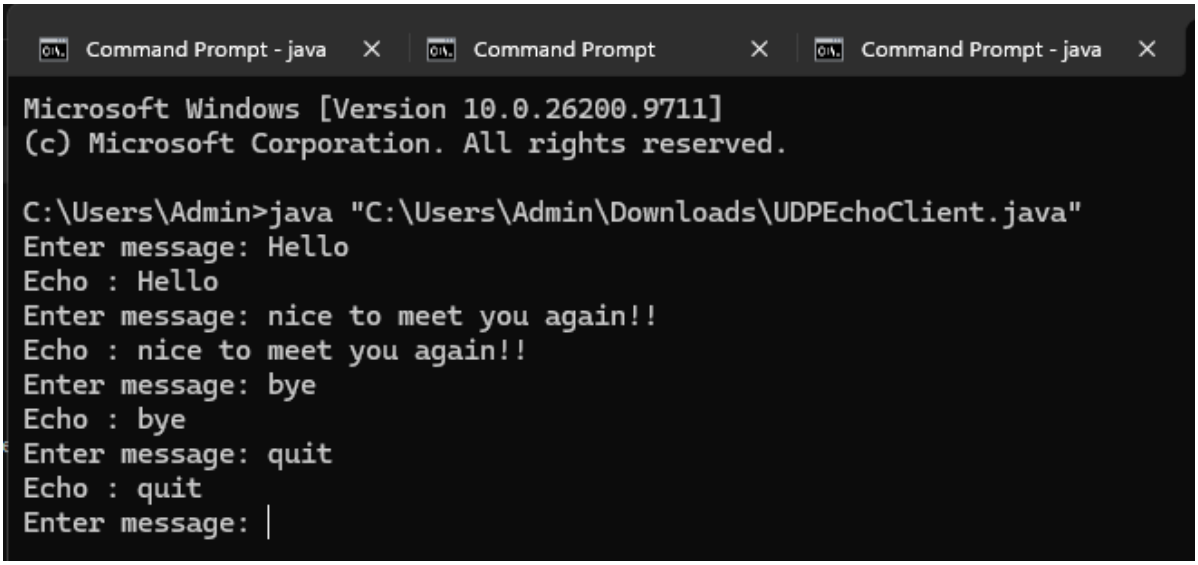


Fig 4. UDP Echo Client – messages sent and echoed back by the server

RESULT :

Thus, the echo client–server application was successfully implemented using TCP and UDP sockets in Java, and the server echoed back every message sent by the client.